

Clase 019 — Expresiones regulares para análisis de logs y datos

Parte: **0 — Fundamentos y prerequisites** · Fuente: *Friedl, Mastering Regular Expressions (O'Reilly)* 🕒
Duración estimada: **100 min** · Nivel: **Fundamentos**

Objetivo

Dominar las expresiones regulares para extraer, validar y correlacionar información en logs, capturas y volcados de datos. Las regex son omnipresentes en SIEM, IDS, grep y herramientas de análisis; saber escribirlas bien multiplica tu velocidad como analista.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Construir** patrones con clases, cuantificadores y anclas.
- 2. **Capturar** subcadenas con grupos y referencias.
- 3. **Extraer** IOCs (IPs, hashes, URLs, correos) de texto.
- 4. **Aplicar** regex en `grep -P`, Python (`re`) y herramientas de análisis.
- 5. **Evitar** patrones peligrosos (ReDoS) y errores frecuentes.

Temas

#	Tema	Por qué importa
1	Literales y metacaracteres	La base de todo patrón
2	Clases de caracteres	<code>[...]</code> , <code>\d</code> , <code>\w</code> , <code>\s</code>
3	Cuantificadores	<code>*</code> , <code>+</code> , <code>?</code> , <code>{n,m}</code>
4	Anclas y límites	<code>^</code> , <code>\$</code> , <code>\b</code>
5	Grupos y captura	<code>(...)</code> , alternancia, no captura
6	Codicia y pereza	<code>.*</code> vs. <code>.*?</code>
7	Extracción de IOCs	IPs, hashes, dominios
8	ReDoS	Regex que se cuelgan

Definiciones y características

- **Metacarácter**: símbolo con significado especial (`.` `*` `+` `?` `[ ]` `( )` `^` `$` `\`). Clave: escapar con `\` para tratarlo como literal.

- **Clase de caracteres:** conjunto entre corchetes (`[a-f0-9]`). Clave: `\d` = dígito, `\w` = alfanumérico+`_`, `\s` = espacio.
- **Cuantificador codicioso:** `*` / `+` intentan capturar lo máximo. Clave: `.*?` (perezoso) captura lo mínimo; crucial para no "tragar" de más.
- **Grupo de captura:** `( ... )` guarda lo coincido para reutilizarlo. Clave: `(?: ... )` agrupa sin capturar.
- **Ancla:** `^` (inicio), `$` (fin), `\b` (límite de palabra). Clave: evita coincidencias parciales no deseadas.
- **ReDoS:** denegación de servicio por regex con retroceso catastrófico. Clave: cuantificadores anidados sobre patrones ambiguos son peligrosos.

Herramientas y preparación

Practica con `grep -P` (PCRE), Python `re`, y un entorno visual como **regex101.com** (elige el motor PCRE/Python) para ver el árbol de coincidencias. Ten a mano un log real (auth.log, access.log) y una muestra de texto con IOCs para extraer.

Laboratorio guiado

1. **Clases y cuantificadores.** Encuentra todas las horas `HH:MM:SS` en un log:

```
bash grep -oP '\b\d{2}:\d{2}:\d{2}\b' auth.log | head
```

2. **Extraer IPv4.** Patrón razonable para direcciones:

```
bash grep -oP '\b(?:\d{1,3}\.){3}\d{1,3}\b' access.log | sort | uniq -c | sort -nr
```

3. **Grupos de captura en Python:**

```
python import re m = re.search(r'Failed password for (\w+) from ([\d.]+)', linea) if m:
usuario, ip = m.group(1), m.group(2)
```

4. **Extraer hashes.** Distingue MD5 (32 hex) de SHA-256 (64 hex):

```
bash grep -oP '\b[a-fA-F0-9]{64}\b' muestra.txt # SHA-256
```

5. **Correos y URLs.** Escribe patrones para extraer direcciones de correo y URLs `http(s)`.
6. **Codicia vs. pereza.** Compara `<.*>` y `<.*?>` sobre `<a><b>` y observa la diferencia.
7. **Validar en regex101:** pega tus patrones, revisa el desglose y mide pasos de retroceso.

Ejercicios

1. Escribe una regex que extraiga solo IPs privadas (10/8, 172.16/12, 192.168/16) de un log.
2. Captura usuario e IP de cada línea de "Failed password" y cuenta intentos por IP.
3. Crea un patrón que valide un email de forma razonable (sin buscar perfección RFC).
4. Extrae todas las URLs de una página guardada y quédate solo con el dominio.
5. Diferencia con una sola pasada MD5, SHA-1 y SHA-256 por su longitud.
6. Investiga un ejemplo de ReDoS y reescríbelo para hacerlo seguro.

Reto verificable

Escribe `iocextract.py`, una herramienta que reciba un archivo de texto y extraiga y clasifique IOCs: IPv4, dominios, URLs, correos y hashes (MD5/SHA-1/SHA-256), imprimiendo un recuento por categoría y la lista deduplicada. Usa grupos con nombre y evita patrones vulnerables a ReDoS.

Criterio de aceptación: sobre una muestra con IOCs conocidos, la herramienta los extrae todos sin falsos positivos evidentes, clasifica correctamente los tres tipos de hash por longitud, y deduplica. El código no contiene cuantificadores anidados peligrosos. Verificable contando manualmente los IOCs de la muestra.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El patrón captura de más	Cuantificador codicioso. Usa versión perezosa <code>*?</code> o clases más específicas.
<code>.</code> no coincide con lo esperado	<code>.</code> no incluye salto de línea por defecto. Usa flag <code>DOTALL</code> si lo necesitas.
Falsos positivos en IPs (999.999.999.999)	El patrón no valida rangos 0-255. Refínalo o valida después con código.
grep no entiende <code>\d</code>	grep básico no es PCRE. Usa <code>grep -P</code> o <code>-E</code> con clases POSIX.
La regex cuelga el proceso	ReDoS por retroceso. Simplifica y evita <code>(a+)+</code> sobre entradas largas.

Preguntas frecuentes

? ¿Debo validar IPs solo con regex? La regex acota candidatos, pero validar rangos 0-255 exactos con regex es engorroso. Es mejor extraer con regex y validar con código (`ipaddress` en Python).

? ¿Por qué mi regex funciona en regex101 pero no en grep? Los motores difieren. grep básico usa BRE/ERE; PCRE (regex101/Python) admite `\d`, lookarounds, etc. Ajusta al motor real.

? ¿Qué es un lookahead/lookbehind? Aserciones que comprueban contexto sin consumirlo (`(?= ... )`, `(?<= ... )`). Útiles para extraer algo "seguido de" o "precedido por" sin incluirlo.

? ¿Regex sirve para parsear HTML/JSON? Para extracciones puntuales sí, pero para estructuras anidadas usa parsers dedicados. La regex no maneja bien anidamiento arbitrario.

Referencias

- Jeffrey Friedl, *Mastering Regular Expressions* (O'Reilly).
- Python `re` — <https://docs.python.org/3/library/re.html>
- regex101 (probador interactivo) — <https://regex101.com/>
- OWASP: Regular Expression DoS (ReDoS) — https://owasp.org/www-community/attacks/Regular_expression_Denial_of_Service_-_ReDoS

Siguiente clase

Clase 020 - Sistemas de numeracion y encoding: binario, hex, base64 y URL